Listing 4.28 Advanced `bool` query with `term` queries

```
GET emails/_search
{
  "query": {
    "bool": {
      "must": [
        {"term": { "attachments.filename.keyword": "file2.txt"}},
        {"term": { "attachments.filetype.keyword": "private" }}
      ]
    }
  }
}
```

A bool query that searches for a match with a filename and file type

must clause defining the mandatory clauses

Defines the query as a bool query

When this query is executed, it returns the following document:

```
"hits" : [[
{
  ...
  "_source" : {
    "to:" : "mrs.doe@johndoe.com",
    "subject" : "Multi attachments test",
    "attachments" : [
    {
      "filename" : "file2.txt",
      "filetype" : "confidential"
    },
    ..
   ]
 }
}]
```

Unfortunately, this result is incorrect—no document exists with the combination of the name file2.txt and the type `private` (recheck table 4.4). This is where the `object` data type breaks down—it can't honor the relationships between the inner objects.

Ideally, the values `file2.txt` and `private` are in different objects, so the search shouldn't consider them a single entity. The reason is that inner objects are not stored as individual documents—they are flattened:

```
{
  ...
  "attachments.filename" :["file1.txt","file2.txt","file3.txt"]
  "attachments.filetype":["private","confidential"]
}
```

As you can see, `filenames` are collected as an array and stored in the `attachments.filename` field, so they are also file types. Unfortunately, because they are stored this way, their relationship is lost. We can't say if file1.txt is `private` or `confidential` because the array doesn't hold the state.

This is the limitation of indexing an array of objects for a field and trying to search the objects as individual documents. The good news is that a data type called `nested` solves this problem, as we discuss in the next section.